

Gerenciador de Tarefas de uma Equipe com Lista Duplamente Encadeada, Fila e Pilha

Thailton Veloso da Silva Luz, Marcos Vinícius Soares da Costa, Marcos Renan Costa Paiva, Júlio Henrique Marta de Lima

Instituto Federal de Educação, Ciência e Tecnologia do Piauí (IFPI)
Campus Picos - Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas

Disciplina: Estrutura de Dados I - Professor: Rogerio Figueredo de Sousa
Picos - PI - Brasil

***Abstract.** This work describes a team task manager developed in C for terminal use. Tasks are stored in a doubly linked list, the weekly plan uses a queue, and recently completed tasks are kept in a stack. The program also has sequential search, Bubble Sort, dynamic memory allocation, and a general report. The source code was divided into modules so that each data structure and group of functions can be studied separately.*

***Resumo.** Neste trabalho, desenvolvemos em Linguagem C um gerenciador de tarefas para uma equipe. O programa funciona pelo terminal e permite cadastrar, editar, buscar, ordenar e remover tarefas. Para aplicar o conteúdo da disciplina, usamos uma lista duplamente encadeada, uma fila e uma pilha. O código foi separado em módulos, e os nós das estruturas são criados dinamicamente e liberados quando deixam de ser necessários.*

1. Introdução

Escolhemos desenvolver um gerenciador de tarefas porque esse tema permite usar as estruturas de dados em uma situação fácil de entender. Em uma equipe, cada atividade precisa ter um responsável, uma prioridade, um prazo e um status. O programa reúne essas informações e permite acompanhar o que está pendente, em andamento ou concluído.

Durante o desenvolvimento, usamos os conteúdos vistos em Estrutura de Dados I. A lista, a fila e a pilha foram implementadas pelo próprio grupo, sem usar uma biblioteca pronta. Também trabalhamos com ponteiros, busca sequencial, Bubble Sort, malloc e free.

Nosso objetivo foi fazer um sistema pequeno, mas completo para a proposta do trabalho. Neste relatório, explicamos como ele funciona, onde cada estrutura foi usada e quais dificuldades apareceram durante a implementação.

2. Funcionamento do Sistema

Cada tarefa possui código, título, descrição, responsável, prioridade, prazo e status. O código não pode se repetir e é usado para localizar a tarefa nas operações do sistema. Esses dados ficam reunidos na estrutura `Tarefa`.

Pelo menu, o usuário pode cadastrar, editar, remover e listar tarefas. Também pode montar uma fila de atividades para a semana, iniciar a próxima tarefa, alterar o

status e reabrir a última tarefa concluída. As outras opções fazem busca, ordenação e mostram um relatório geral. Incluímos validações para números, códigos repetidos e datas inválidas.

A lista guarda todos os dados da tarefa. Já a fila e a pilha guardam apenas o código. Fizemos assim para não repetir título, responsável, prazo e outros campos em três lugares. Quando uma tarefa é removida, seu código também é retirado da fila e da pilha.

2.1. Organização em Módulos

Para evitar um arquivo muito grande, dividimos o código por responsabilidade. O arquivo `main.c` apenas inicializa as estruturas, controla as opções e libera a memória no final. A leitura e a validação ficam em `entrada.c`, e as funções auxiliares da tarefa ficam em `tarefa.c`.

As estruturas foram separadas em `lista.c`, `fila.c` e `pilha.c`. As funcionalidades também foram divididas: `cadastro.c` contém cadastro, edição, remoção e listagem; `planejamento.c` cuida da fila semanal e dos status; e `consultas.c` reúne busca, ordenação e relatório. Assim ficou mais fácil encontrar e testar cada parte.

3. Estruturas de Dados e Algoritmos

A Tabela 1 mostra de forma resumida onde cada estrutura foi usada. Nas complexidades, a letra n representa a quantidade de tarefas cadastradas.

Tabela 1. Relação entre as estruturas e sua aplicação no sistema

Elemento	Aplicação no projeto	Principais custos
Lista duplamente encadeada	Cadastro principal e navegação das tarefas nos dois sentidos.	Inserção no fim $O(1)$; busca $O(n)$
Fila encadeada	Tarefas pendentes selecionadas para execução durante a semana.	Enfileirar e desenfileirar $O(1)$
Pilha encadeada	Histórico das tarefas concluídas recentemente e reabertura da última.	Empilhar e desempilhar $O(1)$
Busca sequencial	Consulta por código, título, responsável ou palavra-chave.	$O(n)$
Bubble Sort	Ordenação por prioridade, prazo ou status.	$O(n^2)$

3.1. Lista Duplamente Encadeada

A lista é a estrutura principal do programa. Ela é representada por `ListaTarefas`, que guarda ponteiros para o primeiro e o último nó. Cada `NoTarefa` possui uma tarefa, um ponteiro para o nó anterior e outro para o próximo. Segundo Black (2022), esse tipo de encadeamento permite percorrer a lista nos dois sentidos. No sistema, usamos essa característica para listar do início ao fim ou do fim ao início.

A função `inserirNoFim()` cria um novo nó com `malloc` e faz as ligações com o último elemento. Como já existe um ponteiro para o fim, essa inserção tem custo

O(1). Para buscar um código, é necessário percorrer os nós, por isso a busca tem custo $O(n)$.

3.2. Fila de Execução Semanal

A fila é armazenada em `FilaSemana`. Conforme a definição apresentada por Black (2020), uma fila segue a ordem FIFO: o primeiro item que entra é o primeiro que sai. A função `enfileirar()` coloca o código no final, enquanto `desenfileirar()` retira o código do início.

Na prática, essa fila representa o planejamento da semana. Somente tarefas pendentes podem ser adicionadas, e o programa impede que o mesmo código entre duas vezes. Ao iniciar a próxima tarefa, ela sai da fila e passa para o status em andamento.

3.3. Pilha de Tarefas Concluídas

A pilha é representada por `PilhaConcluidas` e mantém um ponteiro para o topo. Uma pilha segue a ordem LIFO, ou seja, o último item inserido é o primeiro removido. Ao concluir uma tarefa, a função `empilhar()` coloca seu código no topo. A opção de reabertura usa `desempilhar()` para recuperar a tarefa concluída mais recentemente e mudar seu status para pendente.

3.4. Busca e Ordenação

A função `buscarNoPorCodigo()` percorre a lista até encontrar o código informado. A busca por título, responsável ou palavra-chave também verifica os nós um por um. Por isso, no pior caso, todas as tarefas precisam ser analisadas.

Para a ordenação, usamos o Bubble Sort na função `ordenarLista()`. Ele compara elementos vizinhos e realiza trocas até a lista ficar na ordem escolhida. O NIST apresenta o Bubble Sort como um algoritmo simples de trocas sucessivas e informa complexidade $O(n^2)$ no caso geral. No projeto, ele ordena por prioridade, prazo ou status.

4. Fluxo de Uso e Gerenciamento de Memória

O uso mais comum começa com o cadastro das tarefas. Depois, as tarefas pendentes escolhidas para a semana entram na fila. Quando a primeira é iniciada, ela sai da fila e fica em andamento. Ao ser concluída, seu código entra na pilha. Se for necessário voltar atrás, a última tarefa concluída pode ser reaberta.

O relatório geral foi feito com um percurso pela lista. Ele mostra o total por status, quantas tarefas estão na fila da semana e quantas pertencem a cada responsável.

Os nós da lista, fila e pilha são criados dinamicamente com `malloc`. Na remoção de tarefas e na retirada de elementos da fila ou da pilha, a memória é liberada com `free`. Antes do encerramento, as funções `liberarLista()`, `liberarFila()` e `liberarPilha()` percorrem o que restou e liberam todos os nós. O manual GNU recomenda usar `free` quando um bloco criado com `malloc` não for mais necessário.

5. Testes, Resultados e Dificuldades

O programa foi compilado com GCC no padrão C11 e com as opções `-Wall`, `-Wextra` e `-Wpedantic`, sem avisos. Também usamos a análise estática com `-fanalyzer` e os sanitizadores de endereço e comportamento indefinido.

Para testar o fluxo, cadastramos três tarefas com responsáveis, prioridades, prazos e status diferentes. Depois, percorremos a lista nos dois sentidos, colocamos tarefas na fila, iniciamos a primeira, concluímos e reabrimos pela pilha. Também testamos busca, ordenação, edição, relatório e remoção.

Uma das partes que exigiu mais atenção foi manter as estruturas sincronizadas. Como a fila e a pilha guardam códigos, ao remover uma tarefa da lista também precisamos apagar esse código das outras estruturas. Outra dificuldade foi validar as entradas sem deixar o programa encerrar por causa de um valor incorreto. Depois desses ajustes, os testes não indicaram acesso inválido à memória.

6. Considerações Finais

Com este trabalho, conseguimos perceber melhor a diferença entre lista, fila e pilha. Antes elas eram vistas principalmente na teoria, mas no programa cada uma recebeu uma função clara: a lista mantém o cadastro, a fila organiza a ordem da semana e a pilha permite recuperar a tarefa concluída mais recentemente.

O resultado final atende ao que foi proposto: funciona pelo terminal, possui busca, ordenação, relatório, alocação dinâmica e três estruturas encadeadas. A divisão em módulos também ajudou a entender o papel de cada arquivo. Em uma versão futura, poderíamos salvar as tarefas em arquivo, mas preferimos concluir primeiro as estruturas exigidas no trabalho.

Referências

- Black, P. E. (2019). Stack. In: Dictionary of Algorithms and Data Structures. National Institute of Standards and Technology. Disponível em: <https://www.nist.gov/dads/HTML/stack.html>. Acesso em: 5 ago. 2026.
- Black, P. E. (2020). Queue. In: Dictionary of Algorithms and Data Structures. National Institute of Standards and Technology. Disponível em: <https://www.nist.gov/dads/HTML/queue.html>. Acesso em: 5 ago. 2026.
- Black, P. E. (2022). Linked list. In: Dictionary of Algorithms and Data Structures. National Institute of Standards and Technology. Disponível em: <https://www.nist.gov/dads/HTML/linkedList.html>. Acesso em: 6 ago. 2026.
- Black, P. E. (2023). Bubble sort. In: Dictionary of Algorithms and Data Structures. National Institute of Standards and Technology. Disponível em: <https://www.nist.gov/dads/HTML/bubblesort.html>. Acesso em: 7 ago. 2026.
- Free Software Foundation (2026). Dynamic Memory Allocation. GNU C Language Manual. Disponível em: https://www.gnu.org/software/c-intro-and-ref/manual/html_node/Dynamic-Memory-Allocation.html. Acesso em: 8 ago. 2026.